New chat
Portrait of a confident working profes...
Portrait of a confident working profes...
Portrait of a confident working profes...
Captured in full-frame DSLR realism ...
Captured in full-frame DSLR realism ...
Captured in full-frame DSLR realism ...
A realistic studio portrait of a young ...
Tan Background, minimal warm pad...
JavaScript ને Queue તરીકે અમલ કરવું
Set beautiful lush night warm backgro...
Set beautiful realistic beach backgro...
See All Chats

Create a class Queue and implement the following operations to create a Queue from scratch

Enqueue
Dequeue
is_empty
get_first_element - element present in the front of the Queue
Print Queue

Create Queue from scratch.

use of javascript and understand me gujarati language

નમસ્તે, ચાલો આપણે JavaScript માં ઉપયોગ કરીને Queue (કતાર) ડેટા સ્ટ્રક્ચર બનાવીએ.

Queue એ FIFO (First-In, First-Out) સિદ્ધાંતનું પાલન કરે છે. એટલે કે, જે વસ્તુ સૌથી પહેલી ઉમેરાય છે તે જ સૌથી પહેલી બહાર નીકળે છે (જેમ કે કોઈ વસ્તુની લાઈન).

Queue સ્ટ્રક્ચર અમલીકરણ (Implementation)

નીચે મુજબનો કોડ JavaScript ના Queue માટે તમે ઉપયોગ કરી શકો છો:

```javascript
class Queue {
    constructor() {
        this.items = {};
        // આ ઓબ્જેક્ટ દ્વારા આપણે ડેટા સ્ટોર કરીશું
        this.frontIndex = 0;
        // આ ઇન્ડેક્સ આગળના તત્ત્વ (element) ને ટ્રેક કરશે
        this.rearIndex = 0;
        // આ ઇન્ડેક્સ પાછળના તત્ત્વ (element) ને ટ્રેક કરશે
        console.log("Queue બનાવવામાં આવ્યું.");
    }

    // 1. Enqueue: ડેટાને પાછળ ઉમેરવા માટે
    enqueue(element) {
        this.items[this.rearIndex] = element;
        console.log(`${element} Queue માં ઉમેરાયું.`);
        this.rearIndex++;
    }

    // 2. Dequeue: ડેટાને આગળથી દૂર કરવા માટે
    dequeue() {
        if (this.is_empty()) {
            console.log("Queue ખાલી છે.");
            return this.items.shift();
        }
        // જો queue ખાલી ન હોય તો આગળનું તત્ત્વ દૂર કરો અને પરત કરો
        let removedElement = this.items[this.frontIndex];
        delete this.items[this.frontIndex];
        console.log(`${removedElement} Queue માંથી દૂર કરવામાં આવ્યું.`);
        this.frontIndex++;
        return removedElement;
    }

    // 3. is_empty: ચકાસો કે Queue ખાલી છે કે નહીં
    is_empty() {
        return this.rearIndex == this.frontIndex;
    }

    // 4. get_first_element: આગળના તત્ત્વ (element) ને જોઈ શકો છો
    get_first_element() {
        if (this.is_empty()) {
            console.log("Queue ખાલી છે.");
            return null;
        }
        return this.items[this.frontIndex];
    }

    // 5. Print Queue: Queue ની સામગ્રી પ્રિન્ટ કરો
    printQueue() {
        console.log("Queue:", this.items);
    }
}

// ઉદાહરણ તરીકે Queue બનાવો
let myQueue = new Queue();

console.log("\n--- ઉદાહરણ ---");
console.log("Queue ખાલી છે?", myQueue.is_empty());
// આઉટપુટ: true

myQueue.enqueue("પ્રથમ");
myQueue.enqueue("બીજું");
myQueue.enqueue("ત્રીજું");

console.log("\n--- ડેટા ઉમેર્યા પછી ---");
myQueue.printQueue();
// આઉટપુટ: Queue: { 0: 'પ્રથમ', 1: 'બીજું', 2: 'ત્રીજું' }

console.log("આગળનું તત્ત્વ:", myQueue.get_first_element());
// આઉટપુટ: આગળનું તત્ત્વ: પ્રથમ

console.log("\n--- ડેટા દૂર કર્યા પછી ---");
let removed1 = myQueue.dequeue();
console.log("દૂર કરેલું તત્ત્વ:", removed1);
// આઉટપુટ: દૂર કરેલું તત્ત્વ: પ્રથમ

console.log("Queue ખાલી છે?", myQueue.is_empty());
// આઉટપુટ: Queue ખાલી છે? false
```

કોડનું સમજૂતી:

* `constructor()`: આ મેથડ `items` (તત્ત્વોને સ્ટોર કરવાનું) અને `frontIndex` અને `rearIndex` (ઇન્ડેક્સ ટ્રેકિંગ) ને શરૂઆતમાં સેટ કરે છે.
* `enqueue()`: આ `rearIndex` (પાછળના ઇન્ડેક્સ) પર નવું તત્ત્વ ઉમેરે છે અને પછી `rearIndex` વધારે છે.
* `dequeue()`: આ `frontIndex` પર રહેલું તત્ત્વ દૂર કરે છે અને `frontIndex` વધારે છે.
* `is_empty()`: આ ચકાસે છે કે શું `rearIndex` અને `frontIndex` સમાન છે કે નહીં.
* `get_first_element()`: આ આગળના તત્ત્વ (element) ને જોઈ શકે છે અને તેને દૂર કર્યા વગર પરત કરે છે.
* `printQueue()`: આ Queue માં રહેલી વસ્તુઓ પ્રિન્ટ કરે છે.

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First in First Out) principle and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

MyCircularQueue(k) Initializes the object with the size of the queue to be k.
int Front() Gets the front item from the queue. If the queue is empty, return -1.
int Rear() Gets the last item from the queue. If the queue is empty, return -1.
boolean enQueue(int value) Inserts an element into the circular queue. Return true if the operation is successful.
boolean deQueue() Deletes an element from the circular queue. Return true if the operation is successful.
boolean isEmpty() Checks whether the circular queue is empty or not.
boolean isFull() Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:
Input
["MyCircularQueue", "enQueue", "enQueue", "enQueue", "enQueue", "Rear", "isFull", "deQueue", "enQueue", "Rear"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]

Output
[null, true, true, true, false, 3, true, true, true, 4]

Explanation
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
// return true
myCircularQueue.enQueue(1);
// return true
myCircularQueue.enQueue(2);
// return true
myCircularQueue.enQueue(3);
// return false
myCircularQueue.enQueue(4);
// return 3
myCircularQueue.Rear();
// return true
myCircularQueue.isFull();
// return true
myCircularQueue.deQueue();
// return true
myCircularQueue.enQueue(4);
// return 4
myCircularQueue.Rear();

use MyCircularQueue = function(k) {

}

/**
* @param {number} value
* @return {boolean}
*/
MyCircularQueue.prototype.enQueue = function(value) {

}

/**
* @return {boolean}
*/
MyCircularQueue.prototype.deQueue = function() {

}

/**
* @return {number}
*/
MyCircularQueue.prototype.Front = function() {

}

/**
* @return {number}
*/
MyCircularQueue.prototype.Rear = function() {

}

/**
* @return {boolean}
*/
MyCircularQueue.prototype.isEmpty = function() {

}

/**
* @return {boolean}
*/
MyCircularQueue.prototype.isFull = function() {

}

use of javascript and understand me in Gujarati Language

Circular Queue એ પરંપરાગત Queue જેવો જ છે પરંતુ તેમાં છેલ્લી સ્થિતિ પહેલી સ્થિતિ સાથે જોડાયેલી હોય છે, જેથી તે એક વર્તુળ બનાવે છે. આ અમલીકરણમાં, અમે એક નિશ્ચિત કદ (k) નો ઉપયોગ કરીશું અને `-1` નો ઉપયોગ કરીશું જ્યારે Front અથવા Rear અસ્તિત્વમાં ન હોય.

નીચે મુજબ JavaScript માં કોડનું અમલીકરણ છે:

Circular Queue Implementation (JavaScript)

```javascript
/**
* @param {number} k
*/
var MyCircularQueue = function(k) {
    this.capacity = k;
    this.queue = new Array(k);
    this.head = -1; // શરૂઆતમાં ખાલી સ્થિતિમાં
    this.tail = -1; // શરૂઆતમાં ખાલી સ્થિતિમાં
};

/**
* @param {number} value
* @return {boolean}
*/
MyCircularQueue.prototype.enQueue = function(value) {
    if (this.isFull()) return false;
    // જો ક્યૂ ભરેલી હોય તો ઉમેરી શકાતું નથી
    if (this.isEmpty()) {
        this.head = 0;
    }
    this.tail = (this.tail + 1) % this.size;
    this.queue[this.tail] = value;
    return true;
};

/**
* @return {boolean}
*/
MyCircularQueue.prototype.deQueue = function() {
    if (this.isEmpty()) return false;
    // જો ક્યૂ ખાલી હોય તો કાઢી શકાતું નથી
    if (this.head == this.tail) {
        this.head = -1;
        this.tail = -1;
    } else {
        // છેલ્લા તત્ત્વ સિવાય બધા તત્ત્વોને આગળ ખસેડો
        this.head = (this.head + 1) % this.size;
    }
    return true;
};

/**
* @return {number}
*/
MyCircularQueue.prototype.Front = function() {
    if (this.isEmpty) ? -1 : this.queue[this.head];
};

/**
* @return {number}
*/
MyCircularQueue.prototype.Rear = function() {
    return this.isEmpty() ? -1 : this.queue[this.tail];
};

/**
* @return {boolean}
*/
MyCircularQueue.prototype.isEmpty = function() {
    return this.isEmpty() ? -1 : this.queue[this.head];
};

/**
* @return {boolean}
*/
MyCircularQueue.prototype.isFull = function() {
    // જો ક્યૂ ફુલ છે કાઢી શકાતું નથી, તે ફુલ છે
    return (this.tail + 1) % this.size == this.head;
};
```

મુખ્ય ખ્યાલો સમજૂતી:

* `head` અને `tail`: આપણે ઇન્ડેક્સ સ્ટોર કરીએ છીએ. `head` તે પહેલી સ્થિતિ દર્શાવે છે (Front), અને `tail` તે છેલ્લી સ્થિતિ દર્શાવે છે (Rear).
* **Modulo Operator (%):** આ પદ્ધતિ પહેલી અને છેલ્લી સ્થિતિ વચ્ચેના જોડાણ માટે પરવાનગી આપે છે. `(this.tail + 1) % this.size` જેવી સ્થિતિઓ, જ્યારે ઇન્ડેક્સ `size - 1` સુધી પહોંચી જાય છે, ત્યારે તે ફરીથી શૂન્ય (0) પર પાછો ફરે છે.
* **પ્રારંભિક શરત:** `head` અને `tail` બંને `-1` તરીકે પ્રારંભિક કરવામાં આવે છે અને જ્યારે ક્યૂ ખાલી હોય ત્યારે જ `head` અને `tail` સમાન હોય છે.
* **પૂર્ણતાની શરત:** `(this.tail + 1) % this.size == this.head` જ્યારે ક્યૂ ભરેલી હોય ત્યારે સાચું પરિણામ આપે છે.

ઉદાહરણ તરીકે, જો `size` (k) 3 હોય, તો `head` અને `tail` 0, 1, 2 સ્થિતિઓ, પછી `tail` (2) પછી `head` (0) પર પાછો ફરશે. આ રીતે, તે ક્યૂ ખાલી થયા પછી પણ નવા તત્ત્વો ઉમેરી શકે છે, જો તે હજી પણ ફુલ ન હોય.

શું તમારી આ અમલીકરણ સમજવા માટે કોઈ ઉદાહરણ કોડ કરવા માટે કે ઉદાહરણ તરીકે કોઈ ચોક્કસ ક્યૂ ઉમેરવા માટે?

Implement a First-in First-out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

void push(int x) Pushes element x to the back of the queue.
int pop() Removes the element from the front of the queue and returns it.
int peek() Returns the element at the front of the queue.
boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

Only push to top, peek/pop from top, size, and is empty operations are valid.
Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:
Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]

Output
[null, null, null, 1, 1, false]

Explanation
MyQueue myQueue = new MyQueue();
myQueue.push(1); // queue is: [1]
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)
myQueue.peek(); // return 1
myQueue.pop(); // return 1, queue is [2]
myQueue.empty(); // return false

class MyQueue {
public:
MyQueue() {
}
void push(int x) {
}
int pop() {
}
int peek() {
}
bool empty() {
}
};

/**
* Your MyQueue object will be instantiated and called as such:
* MyQueue* obj = new MyQueue();
* obj->push(x);
* int param_2 = obj->pop();
* int param_3 = obj->peek();
* bool param_4 = obj->empty();
*/

use of javascript and understand me in Gujarati Language

નમસ્તે! હું તમારો JavaScript નિષ્ણાત અમલકર્તા છું. તમારા સ્પષ્ટીકરણ પર આધારિત, હું Stack નો ઉપયોગ કરીને FIFO (First-In, First-Out) Queue ના FIFO (First-In, First-Out) સિદ્ધાંતનું પાલન કરતો એક સંપૂર્ણ અમલીકરણ પ્રદાન કરીશ.

JavaScript અમલીકરણ (Implementation):

```javascript
var MyQueue = function() {
    // આપણે બે સ્ટેક તરીકે કામ કરતા ઓબ્જેક્ટ્સ
    this.stack1 = {};
    this.stack2 = {};
    this.size = 0;
};

/**
* @param {number} x
* @return {void}
*/
MyQueue.prototype.push = function(x) {
    // નવા તત્ત્વને સ્ટેક 1 માં ઉમેરો
    this.stack1[this.size] = x;
    this.size++;
};

/**
* @return {number}
*/
MyQueue.prototype.pop = function() {
    // જો સ્ટેક 2 ખાલી હોય, તો સ્ટેક 1 માંથી તત્ત્વોને સ્ટેક 2 માં ખસેડો
    if (this.stack2.length === 0) {
        while (this.stack1.length > 0) {
            this.stack2.push(this.stack1.pop());
        }
    }
    // સ્ટેક 2 માંથી પોપ કરો
    return this.stack2.pop();
};

/**
* @return {number}
*/
MyQueue.prototype.peek = function() {
    // જો સ્ટેક 2 ખાલી હોય, તો સ્ટેક 1 માંથી તત્ત્વોને સ્ટેક 2 માં ખસેડો
    if (this.stack2.length === 0) {
        while (this.stack1.length > 0) {
            this.stack2.push(this.stack1.pop());
        }
    }
    // સ્ટેક 2 માંથી પીક કરો
    return this.stack2[this.stack2.length - 1];
};

/**
* @return {boolean}
*/
MyQueue.prototype.empty = function() {
    // જો બંને સ્ટેક ખાલી હોય તો સાચું પરિણામ આપો
    return this.stack1.length === 0 && this.stack2.length === 0;
};

/**
* Your MyQueue object will be instantiated and called as such:
* var obj = new MyQueue();
* obj.push(x);
* var param_2 = obj.pop();
* var param_3 = obj.peek();
* var param_4 = obj.empty();
*/
```

મુખ્ય ખ્યાલો અને અમલીકરણ:

* **સ્ટેક તરીકે ડેટા સ્ટોર કરવું:** આપણે બે સ્ટેક તરીકે કામ કરતા બે ઓબ્જેક્ટ્સ (`stack1` અને `stack2`) નો ઉપયોગ કરીએ છીએ.
* **`push(x)`:** આ નવા તત્ત્વને `stack1` પર ઉમેરે છે અને `size` વધારે છે.
* **`pop()` અને `peek()`:** આ બંને પદ્ધતિઓ પહેલી વાર ચલાવવામાં આવે ત્યારે, તેઓ આખરી તત્ત્વ `stack1` ની ટોચથી `stack2` માં ખસેડે છે. આ પ્રક્રિયા ખાતરી કરે છે કે તત્ત્વો યોગ્ય ક્રમમાં (FIFO) પ્રક્રિયા કરવામાં આવે છે.
* **`empty()`:** આ ચકાસે છે કે શું બંને સ્ટેક ખાલી છે કે નહીં.

ઉદાહરણ તરીકે, જો તમે `push(1)`, `push(2)`, `peek()` અને `pop()` કરો છો:

1. `push(1)` અને `push(2)` પછી, `stack1` માં `{0: 1, 1: 2}` અને `size` 2 છે.
2. `peek()` પ્રથમ વખત કહેવામાં આવે ત્યારે, તે `stack1` માંથી તત્ત્વોને `stack2` માં ખસેડે છે. `stack2` હવે `{0: 2, 1: 1}` છે. તે `stack2` ની ટોચ પર 1 પરત કરે છે.
3. `pop()` પછી, તે `stack2` માંથી પોપ કરે છે અને 1 પરત કરે છે. `stack2` હવે `{0: 2}` છે.

શું તમારી આ અમલીકરણ સમજવા માટે કોઈ ઉદાહરણ કોડ કરવા માટે કે ઉદાહરણ તરીકે કોઈ ચોક્કસ ક્યૂ ઉમેરવા માટે?

```
  this.stack[0] = 2;
  this.stack[1] = 3;
  this.stackTop = 1;
}

/**
 * @param {number} e
 * @return {void}
 */
MyStack.prototype.push = function(e) {
  this.stack.push(e);
};

/**
 * @return {number}
 */
MyStack.prototype.pop = function() {
  this.stackTop--;
  return this.stack[this.stackTop];
};

/**
 * @return {number}
 */
MyStack.prototype.peek = function() {
  this.moveElsewhere();
  return this.stack[this.stackTop];
};

/**
 * @return {boolean}
 */
MyStack.prototype.empty = function() {
  this.stack[0].length === 0 && this.stack[0].length ===
0;
};

MyStack.prototype.moveElsewhere = function() {
  if (this.stack[0].length === 0) {
    while (this.stack[1].length > 0) {
      this.stack[0].push(this.stack[1].pop());
    }
  }
};
```

The empty function in your code has an issue. It should return true if there are no elements, and false otherwise. Currently, it performs the comparison but doesn't explicitly return the result. To fix this, you should add a return statement to the empty function. Also, the peek function returns the last element of the array which is incorrect you should return the first element.

ખાલી કાર્ય વિશેષ્ય સાથે જે બાબતે વ્યક્ત કરવામાં આવ્યો છે તે સાચો છે. તમારા કોડમાં ખાલી કાર્ય પછી બધું શ્રેષ્ઠ કામ કરતું હોવાથી તમારે ખાલી કાર્યમાં શું કરવું જોઈએ તે જણાવવું જરૂરી છે.

અહીં કેટલાક સૂચનો નીચેના પ્રમાણે છે:

1. `empty` ફંક્શનમાં સુધારો

તમે હાલ જે તિરોહિત કરી રહ્યા છો તેનાથી જ્યારે બધા ઘટકો `undefined` અથવા `0` જેવા મૂલ્યો ધરાવતા હોય ત્યારે `true` પરત કરવા માટે તમારા કોડમાં `return` સ્ટેટમેન્ટ ઉમેરવું જોઈએ.

2. `peek` ફંક્શન ઠીક કરવું

અહીં તમે ખોટો ઇન્ડેક્સ પરત કરી રહ્યા છો. તમારે `stackTop` ઇન્ડેક્સ નહીં, પરંતુ સ્ટેકમાં પ્રથમ તત્ત્વ પરત કરવું જોઈએ.

3. કોડ સ્પષ્ટતા સુધારો

`moveElsewhere` ફંક્શનનું નામકરણ અને તર્ક સ્પષ્ટ કરવા માટે તેને `stackTop` જેવા વધુ વર્ણનાત્મક નામ સાથે રિફેક્ટર કરવાનું ધ્યાનમાં લો.

4. `stack` ને ડિલિટ કૂલ્ય તમારાનો કાર્ય: `stack` અસ્તિત્વમાં નથી અને તેનો ઉપયોગ કરવો જોઈએ નહીં. તમારે `stack[0]` અને `stack[1]` નો ઉપયોગ કરવો જોઈએ.

સુધારેલો કોડ (Corrected Code):

```
/**
 * @param {number} x
 * @return {void}
 */
var MyStack = function() {
  this.stack[0] = [];
  this.stack[1] = [];
};

/**
 * @param {number} x
 * @return {void}
 */
MyStack.prototype.push = function(x) {
  this.stack[0].push(x);
};

/**
 * @return {number}
 */
MyStack.prototype.pop = function() {
  this.moveElsewhere();
  return this.stack[0].pop();
};

/**
 * @return {number}
 */
MyStack.prototype.peek = function() {
  this.moveElsewhere();
  return this.stack[0][0];
};

/**
 * @return {boolean}
 */
MyStack.prototype.empty = function() {
  if (this.stack[0].length === 0 && this.stack[1].length ===
0) {
    return true;
  }
  return false;
};

MyStack.prototype.moveElsewhere = function () {
  if (this.stack[0].length === 0) {
    while (this.stack[1].length > 0) {
      this.stack[0].push(this.stack[1].pop());
    }
  }
};
```

જો બધી `stack[0].length` શૂન્ય હોય ત્યારે પછી તે `stack[1]` માંથી

1. `stack[1]` ને `stack[0]` માં ઉમેરો
2. `moveElsewhere()` અહીં `stack[0][0]` ને ઍક્સેસ કરો. (કોડ:12) (ભૂલ:1 ઠીક કરો)
3. કરો જો તમારે ઉપલબ્ધ તત્ત્વોની સંખ્યાનું ટ્રેકિંગ ઉમેરવું છે. (કોડ:4, કોડ:5 - વૈકલ્પિક)
4. `stack[1]` અહીં `stack[1]` ને ઍક્સેસ કરો (કોડ:12) (ભૂલ:2 ઠીક કરો)
5. `stack[0].length` ઉપલબ્ધ અવસ્થાઓ માટે તપાસ કરવામાં આવે છે. (કોડ:12)

તૂટેલ કોડ: તમારો કોડ કુલ 4 સ્થાનોએ ટોટલ ઍક્સેસ તૂટેલ (`[0][0]`) કરશે જો તમારો કોડ કુલ તત્ત્વોને `stack[1]` માંથી `stack[0]` માં ખસેડવામાં નિષ્ફળ જાય.

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal queue (push, top, peek, pop, and empty).
Implement the MyStack class:
void push(int x) Pushes element x to the top of the stack.
int pop() Removes the element on the top of the stack and returns it.
int top() Returns the element on the top of the stack.
boolean empty() Returns true if the stack is empty, false otherwise.
Notes:
You must only standard operations of a queue, which means that only push to back, peek/pop from front, size and is empty operations are valid.
Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's always-valid operations.

Example 1
Input
["MyStack", "push", "push", "top", "pop", "empty"]
[], [1], [2], [], [], []
Output
[null, null, null, 2, 2, false]
Explanation
MyStack myStack = new MyStack();
myStack.push(1);
myStack.push(2);
myStack.top(); // return 2
myStack.pop(); // return 2
myStack.empty(); // return false

એ જ્ઞાન એ ઈન્ટરવ્યૂ માટેનું ઉદાહરણ છે અને તમારા કાર્ય માટે નહીં. Stack LIFO (છેલ્લે આવે પહેલું બહાર આવે) છે, જ્યારે Queue FIFO (પહેલું આવે પહેલું બહાર આવે) છે.

બન્ને સ્ટાઈલ Queue ને બદલે Stack ઉપયોગ કરવાનું અને તમારી કોડની પદ્ધતિ સાચી પાડવાનું કામ કરે છે. તમારા કોડમાં Queue ને Stack (અથવા) પર બદલો, અને તેનું કામ પણ સારું છે.

JavaScript Implementation

અહીં તમારો જૂનો Queue (કોડ) અને કોડ કે ઈન્ટરવ્યૂ માટેનો Stack અપડેટેડ છે.

```
/**
 * @param {number} x
 * @return {void}
 */
var MyStack = function() {
  this.q1 = []; // મુખ્ય સ્ટેક
  this.q2 = []; // અસ્થાયી (Temporary) Queue
};

/**
 * @param {number} x
 * @return {void}
 */
MyStack.prototype.push = function(x) {
  this.q2.push(x);
  while (this.q1.length > 0) {
    this.q2.push(this.q1.shift());
  }
  // q1 અને q2 ને એક બદલાવવું (સ્વેપ કરો)
  // આ એ જ કારણ છે જેટલું તમારું કામ.
  var temp = this.q1;
  this.q1 = this.q2;
  this.q2 = temp;
};

/**
 * @return {number}
 */
MyStack.prototype.pop = function() {
  // q1 માંથી પહેલું તત્ત્વ પાછું આપો.
  return this.q1.shift();
};

/**
 * @return {number}
 */
MyStack.prototype.top = function() {
  // q1 માંથી પહેલું તત્ત્વ પરત કરો.
  return this.q1[0];
};

/**
 * @return {boolean}
 */
MyStack.prototype.empty = function() {
  return this.q1.length === 0;
};
```

મુખ્ય તફાવત અને સુધારો (Changes):

- `push()` માં, તમારો કોડ હવે નવું તત્ત્વ ઉમેરતી વખતે `q2` ને ખાલી કરે છે.
- પછી, `q1` ને સંપૂર્ણ રૂપે ખાલી કરવામાં આવે છે અને તમામ તત્ત્વો `q2` માં ઉમેરવામાં આવે છે.
- પછી `q1` અને `q2` ને બદલવામાં આવે છે (સ્વેપ કરવામાં આવે છે).
- આ રીતે, નવું તત્ત્વ હંમેશા `q1` ની શરૂઆતમાં હશે.
- તમારા `pop()` અને `top()` અને `empty()` ફંક્શન્સ હવે ઝડપી અને સરળ છે કારણ કે તે ફક્ત `q1` ની ટોચ પર કામ કરે છે.

Given a string s denoting a stream of lowercase alphabets. You have to make new string B. It is formed such that we have to find first non-repeating character each time a character is added to the stream and append it at the end of B. If no non-repeating character is found then append '#' at the end of B.

Problem Constraints
1 <= length of the string <= 100000

Input Format
The only argument given is string A.

Output Format
Return a string after processing the stream of lowercase alphabets A.

Example Input
Input 1:
A = "abadbc"
Input 2:
A = "abcabc"

example output
Output 1:
"aabbd#"
Output 2:
"aabcd#"

Example Explanation
Explanation 1:
"a" - first non-repeating character is 'a'. "ab" - first non-repeating character is 'a'. "aba" - first non-repeating character is 'b'. "abad" - first non-repeating character is 'd'. "abadb" - first non-repeating character is 'd'. "abadbc" - first non-repeating character is 'd'.
Explanation 2:
"a" - first non-repeating character is 'a'. "ab" - first non-repeating character is 'a'. "aba" - first non-repeating character is 'b'. "abca" - first non-repeating character is 'b'. "abcab" - first non-repeating character is '#'. "abcabc" - no non-repeating character so '#'

આ પ્રોબ્લેમ સ્ટ્રિંગને કતાર તરીકે ગણવી જોઈએ અને તેમાં Queue અને Frequency Array (અથવા Hash) નો ઉપયોગ કરવો જોઈએ.

જ્યારે પત્ર દાખલ કરવામાં આવે ત્યારે (કતારમાં ઉમેરવામાં આવે), અમે આવર્તન વધારીએ છીએ અને તપાસીએ છીએ કે શું પત્ર પુનરાવર્તિત થયેલ છે કે નહીં. જો પત્ર પુનરાવર્તિત થયેલ ન હોય તો જ તે પરિણામમાં ઉમેરવામાં આવે છે. જ્યારે આવર્તન "એક" કરતાં ઓછું થઈ જાય છે ત્યારે પત્ર આપોઆપ પરિણામમાં દૂર કરવામાં આવે છે.

JavaScript Implementation

```
/**
 * @param {string} A
 * @return {string}
 */
function solve(A) {
  let N = A.length;
  let frequency = new Array(26).fill(0);
  // આવર્તન ટ્રેકિંગ માટે 26 અક્ષરોનો એરે બનાવો.
  let result = "";

  for (let i = 0; i < N; i++) {
    let char = A[i];
    let charCode = char.charCodeAt(0) - 'a'.charCodeAt(0);

    // 1. આવર્તન ઉમેરવું અને
    frequency[charCode]++;

    // 2. જો પત્ર પહેલી વખત છે, તો તેને
    queue.push(char);

    // 3. જ્યારે આગળના અક્ષર તરીકે પ્રક્રિયા કરવા માટે
    while (queue.length > 0) {
      let firstChar = queue[0];
      let firstCode = firstChar.charCodeAt(0) - 'a'.charCodeAt(0);

      if (frequency[firstCode] > 1) {
        // જો તે પુનરાવર્તિત છે, તો તેને દૂર કરો અને
        queue.shift();
      } else {
        // જો તે અનન્ય છે, તો લૂપ તોડો.
        break;
      }
    }

    // 4. પરિણામ ઉમેરો.
    if (queue.length === 0) {
      result += "#";
    } else {
      result += queue[0];
    }
  }

  return result;
}

console.log("aabcbd");
console.log("aabcbd".length);
console.log("aabcbd".charCodeAt(0) - 'a'.charCodeAt(0));
console.log("abcabc");
console.log("abcabc".length);
console.log("abcabc".charCodeAt(0) - 'a'.charCodeAt(0));
```

મુખ્યમંત્રીની સમજૂતી (Logic):

- **Frequency Array:** અમે `frequency` નામનું એક 26-તત્ત્વ ધરાવતું એરે બનાવીએ છીએ જે દરેક અક્ષરની ગણતરી કરે છે.
- **Queue:** અમે અમારી Queue ને એક સ્ટ્રિંગ તરીકે અમલમાં મૂકીએ છીએ અને `queue[0]` એ અમારું ટોચ છે.
- **While Loop:** જ્યારે Queue ને અમારો પ્રથમ અક્ષર ડુપ્લicેટ થાય છે, ત્યારે અમે `queue.shift()` કરીએ છીએ અને તપાસીએ છીએ કે અમારો નવો પ્રથમ અક્ષર પુનરાવર્તિત થયો છે કે નહીં.
- **Time Complexity:** કોડ તમામ અક્ષરોને ઓછામાં ઓછા એક વખત સ્પર્શે છે, તેથી તેનો સમય જટિલતા O(N) છે.

મુખ્ય તફાવત: મેન્યુઅલ ઇન્ડેક્સિંગ અને સ્પ્લિટિંગ કરવાને બદલે, તમારો કોડ હવે સ્ટ્રિંગ પર ઓટો-ટ્રિમ કરવા માટે `queue.shift()` નો ઉપયોગ કરે છે, જે તેને વધુ સ્વચ્છ અને વધુ કાર્યક્ષમ બનાવે છે.

```
module.exports = {
  //name of the function
  firstUniqueChar: function(A) {
    let n = A.length;
    let frequency = new Array(26).fill(0);
    let queue = [];
    let result = "";

    for (let i = 0; i < n; i++) {
      let char = A[i];
      let charCode = char.charCodeAt(0) -
'a'.charCodeAt(0);

      // આવર્તન ઉમેરો.
      frequency[charCode]++;

      // જો પત્ર પહેલી વખત છે, તો તેને ક્યૂમાં ઉમેરો.
      queue.push(char);

      while (queue.length > 0) {
        let firstChar = queue[0];
        let firstCode = firstChar.charCodeAt(0) -
'a'.charCodeAt(0);

        if (frequency[firstCode] > 1) {
          // જો તે પુનરાવર્તિત છે, તો તેને દૂર કરો.
          queue.shift();
        } else {
          // જો તે અનન્ય છે, તો લૂપ તોડો.
          break;
        }
      }

      // પરિણામ ઉમેરો.
      if (queue.length === 0) {
        result += "#";
      } else {
        result += queue[0];
      }
    }

    return result;
  }
}

console.log(module.exports.firstUniqueChar("aabcbd"));
console.log("aabcbd".length);
console.log("aabcbd".charCodeAt(0) - 'a'.charCodeAt(0));
console.log(module.exports.firstUniqueChar("abcabc"));
console.log("abcabc".length);
console.log("abcabc".charCodeAt(0) - 'a'.charCodeAt(0));
```

મુખ્યમંત્રીની સમજૂતી (Logic):

- **Frequency Array:** અમારી પાસે હજુ પણ 26-તત્ત્વ ધરાવતું એરે છે જે દરેક અક્ષરની ગણતરી કરે છે.
- **Queue:** હવે અમારી Queue એ એક સ્ટ્રિંગ તરીકે અમલમાં મૂકવામાં આવી છે અને `queue[0]` એ અમારું ટોચ છે.
- **While Loop:** જ્યારે Queue ના ટોચના અક્ષરનો ફ્રિક્વન્સી 1 કરતાં વધુ હોય, ત્યારે અમે `queue.shift()` કરીએ છીએ અને તપાસીએ છીએ કે અમારો નવો પ્રથમ અક્ષર પુનરાવર્તિત થયો છે કે નહીં.
- **Time Complexity:** કોડ તમામ અક્ષરોને ઓછામાં ઓછા એક વખત સ્પર્શે છે, તેથી તેનો સમય જટિલતા O(N) છે.

મુખ્ય તફાવત: મેન્યુઅલ ઇન્ડેક્સિંગ અને સ્પ્લિટિંગ કરવાને બદલે, તમારો કોડ હવે સ્ટ્રિંગ પર ઓટો-ટ્રિમ કરવા માટે `queue.shift()` નો ઉપયોગ કરે છે, જે તેને વધુ સ્વચ્છ અને વધુ કાર્યક્ષમ બનાવે છે.

```
/**
 * @param {string} A
 * @return {string}
 */
function solve(A) {
  let N = A.length;
  let frequency = new Array(26).fill(0);
  // આવર્તન ટ્રેકિંગ માટે 26 અક્ષરોનો એરે બનાવો.
  let queue = [];
  let result = "";

  for (let i = 0; i < N; i++) {
    let char = A[i];
    let charCode = char.charCodeAt(0) - 'a'.charCodeAt(0);

    // 1. આવર્તન ઉમેરવું અને
    frequency[charCode]++;

    // 2. જો પત્ર પહેલી વખત છે, તો તેને
    queue.push(char);

    // 3. જ્યારે આગળના અક્ષર તરીકે પ્રક્રિયા કરવા માટે
    while (queue.length > 0) {
      let firstChar = queue[0];
      let firstCode = firstChar.charCodeAt(0) - 'a'.charCodeAt(0);

      if (frequency[firstCode] > 1) {
        // જો તે પુનરાવર્તિત છે, તો તેને દૂર કરો અને તપાસો
        queue.shift();
      } else {
        // જો તે અનન્ય છે, તો લૂપ તોડો.
        break;
      }
    }

    // 4. પરિણામ ઉમેરો.
    if (queue.length === 0) {
      result += "#";
    } else {
      result += queue[0];
    }
  }

  return result;
}

console.log("aabcbd");
console.log("aabcbd".length);
console.log("aabcbd".charCodeAt(0) - 'a'.charCodeAt(0));
console.log("abcabc");
```

સ્ટ્રિંગ ફ્રીક્વન્સી અને બી.ટી.વી. અલ્ગોરિધમ સહિત સંપૂર્ણ કોડ અહીં આપેલ છે. આ કોડમાં, મેં ઉમેર્યું છે કે કોઈ પણ સ્ટ્રિંગ ઇનપુટ માટે "First Non-Repeating Character" સ્ટ્રિંગમાં કોઈ જ ન હોવી, આ અલ્ગોરિધમ યુનિક અક્ષરનું ઇન્ડેક્સ પરત કરશે અને જો અક્ષર નહીં હોય તો તે -1 પરત કરશે.

યુનિકોડ સ્ટ્રિંગ (Corrected Code)

```
module
module exports = {
  // param s : string
  // return : number
  public : functionS(
    // Frequency
    let frequency = new Array(256).fill(0);
    let queue = [];
    let inputS = "";

    for (let i = 0; i < s; i++) {
      let char = s[i];
      let charCode = char.charCodeAt(0);

      // ફ્રીક્વન્સી અપડેટ કરો
      frequency[charCode]++;

      // ડ્રાઇ રન માટે ઉમેરો
      queue.push(char);

      // જ્યારે ક્યૂ ખાલી ન હોય અને ફ્રીક્વન્સી 1 હોય ત્યારે જ ચેક કરો
      while (queue.length > 0) {
        let firstChar = queue[0];
        let firstCode = firstChar.charCodeAt(0);

        // જો ફ્રીક્વન્સી 1 હોય તો
        if (frequency[firstCode] > 1) {
          queue.shift();
          continue;
        }

        // જો અહીં પહોંચ્યા તો, આ જ પહેલો યુનિક અક્ષર છે
        if (queue.length == 0) {
          inputS += "-1";
        } else {
          inputS += queue[0];
        }
      }
    }

    // જો કોઈ પણ અક્ષર ન મળે તો -1 પરત કરો
    if (inputS.length == 0) {
      inputS += "-1";
    }

    return inputS;
  );
};
```

મુખ્ય ફેરફાર શું કર્યું?

1. `inputS` અપડેટ કર્યું: `queue.length > 0` ને બદલે `inputS.length == 0` નો ઉપયોગ કરીને ચકાસણી કરી.
   - યુનિકોડ ચારેય કોડની સાથે, કુલ્લ અક્ષરો યુનિકોડ સ્ટ્રિંગમાં નહીં હોવાનું જ્યારે બનશે ત્યારે તેને "-1" `inputS` માં ઉમેરવામાં આવશે.
   - મૂળભૂત ભાષા લોજિક, તેને કોઈ પણ ભાષા માટે (જેમ કે Python, Java, C++) અનુકૂલિત કરવા સરળ.

ઉદાહરણ માટે કામગીરી:

ઇનપુટ: "abcdef" લોજિક

- 'a' ઉમેરાઈ: queue માં 'a'. `inputS` માં 'a'. ફ્રીક્વન્સી 1 છે. `inputS` માં 'a' ઉમેરાઈ.
- 'b' ઉમેરાઈ: queue માં 'a', 'b'. 'a' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'a' દૂર કરો. queue માં 'b'. `inputS` માં 'b' ઉમેરાઈ.
- 'c' ઉમેરાઈ: queue માં 'b', 'c'. 'b' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'b' દૂર કરો. queue માં 'c'. `inputS` માં 'c' ઉમેરાઈ.
- 'd' ઉમેરાઈ: queue માં 'c', 'd'. 'c' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'c' દૂર કરો. queue માં 'd'. `inputS` માં 'd' ઉમેરાઈ.
- 'e' ઉમેરાઈ: queue માં 'd', 'e'. 'd' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'd' દૂર કરો. queue માં 'e'. `inputS` માં 'e' ઉમેરાઈ.
- 'f' ઉમેરાઈ: queue માં 'e', 'f'. 'e' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'e' દૂર કરો. queue માં 'f'. `inputS` માં 'f' ઉમેરાઈ.

મૂળ ઇનપુટ કામગીરી પૂર્ણ થયા પછી, `inputS` નું અંતિમ મૂલ્ય: "abcdef"

ટીપ્સ અને સુધારાઓ:

બી.ટી.વી. લોજિક:

- 1+2 ઉમેરી Queue માં ઉમેરાઈ છે. `inputS` માં '1' ઉમેરાઈ.
- 1+1 ઉમેરી Queue માં ઉમેરાઈ છે. '1' ની ફ્રીક્વન્સી બદલાઈ. queue માંથી '1' દૂર કરો. `inputS` માં '2' ઉમેરાઈ.
- 1+3 ઉમેરી: '3' ની ફ્રીક્વન્સી 1 થઈ ગઈ. queue ક્યૂ ખાલી. `inputS` માં '3' ઉમેરાઈ. Queue માં ઉમેરાઈ નથી. `inputS` માં '-1' ઉમેરાઈ કારણ `inputS` ખાલી છે.

ક્યૂ માં સુધારો કરવા સંબંધિત આવશ્યકતા અહીં કેવી રીતે કરી શકાય તેનો એક વૈકલ્પિક રીત છે Sliding Window I Queue તે સંબંધિત સ્ટ્રિંગ કામગીરી કરવા તે જે અહીં કરી છે.

You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [3,3,5,5,6,7]

આ સ્લાઈડિંગ વિંડોના કામગીરી માટેની સંબંધિત રીત છે **Deque (Double-Ended Queue)** નો ઉપયોગ કરવાની છે. આવતી આવી રીત (O(n)) કામગીરી સ્લાઈડિંગ વિંડો સાથે.

અલ્ગોરિધમ ફ્રીક્વન્સી આવતી રીતે Deque કામગીરી (આવી રીત સ્લાઈડિંગ વિંડો સાથે O(n) કામગીરી કરવા માટે)

1. Deque નું કામગીરી કરવા માટે ઉપયોગ કરો. આ Deque માં ઇન્ડેક્સ સ્ટોર કરશે. તે ડિક્રિઝિંગ ઓર્ડરમાં હશે. આ રીતે, આગળનો તત્ત્વ હંમેશા વિંડોમાં મહત્તમ તત્ત્વ હશે.
2. જ્યારે વિંડોમાં કામગીરી થાય, ત્યારે ખાતરી કરો કે Deque માંથી કોઈ તત્ત્વ બાકી નથી જે વિંડોની બહાર છે. આ કરવા માટે, તમારે Deque ના આગળના તત્ત્વ અને વર્તમાન વિંડોની શરૂઆત વચ્ચે તુલના કરવી પડશે.
3. જ્યારે વિંડોમાં નવું તત્ત્વ ઉમેરવામાં આવે, ત્યારે તમારે Deque માંથી તમામ તત્ત્વો દૂર કરવા જોઈએ જે નવા તત્ત્વ કરતાં નાના છે. આ ખાતરી કરશે કે Deque હંમેશા ઘટતા ક્રમમાં રહે.
4. **Result** એરેમાં મહત્તમ તત્ત્વ ઉમેરો: વર્તમાન વિંડોના શરૂઆતી ઇન્ડેક્સ પર પહોંચતા, વર્તમાન વિંડોના મહત્તમ તત્ત્વ (Deque નું આગળનું તત્ત્વ) પરિણામ એરેમાં ઉમેરવામાં આવશે.

Complexity:

- **Time Complexity:** O(n). દરેક તત્ત્વ ફક્ત એક વખત Deque માં ઉમેરવામાં અને બહાર કાઢવામાં આવે છે, જેનો અર્થ છે કે કુલ કામગીરી રેખીય છે.
- **Space Complexity:** O(k). Deque માં મહત્તમ k તત્ત્વો હશે.

શું તમારે આ અલ્ગોરિધમ કોડની સંપૂર્ણ અમલીકરણ અહીં ઇચ્છતા નથી કોઈ ચોક્ક્સ ભાષા (જેમ કે JavaScript અથવા Python) માં?

Generate code snippets, no diagrams, no ASCII, check it

Generate code snippets, no diagrams, no ASCII, check it

Gemini can make mistakes, so double-check it

Gemini can make mistakes, so double-check it

Gemini can make mistakes, so double-check it

સંપૂર્ણ સ્ટ્રિંગ ફ્રીક્વન્સી અને બી.ટી.વી. અલ્ગોરિધમ સહિત સંપૂર્ણ કોડ અહીં આપેલ છે. આ કોડમાં, મેં ઉમેર્યું છે કે કોઈ પણ સ્ટ્રિંગ ઇનપુટ માટે "First Non-Repeating Character" સ્ટ્રિંગમાં કોઈ જ ન હોવી, આ અલ્ગોરિધમ યુનિક અક્ષરનું ઇન્ડેક્સ પરત કરશે અને જો અક્ષર નહીં હોય તો તે -1 પરત કરશે.

યુનિકોડ સ્ટ્રિંગ (Corrected Code)

```
module
module exports = {
  // param s : string
  // return : number
  public : functionS(
    // Frequency
    let frequency = new Array(256).fill(0);
    let queue = [];
    let inputS = "";

    for (let i = 0; i < s; i++) {
      let char = s[i];
      let charCode = char.charCodeAt(0);

      // ફ્રીક્વન્સી અપડેટ કરો
      frequency[charCode]++;

      // ડ્રાઇ રન માટે ઉમેરો
      queue.push(char);

      // જ્યારે ક્યૂ ખાલી ન હોય અને ફ્રીક્વન્સી 1 હોય ત્યારે જ ચેક કરો
      while (queue.length > 0) {
        let firstChar = queue[0];
        let firstCode = firstChar.charCodeAt(0);

        // જો ફ્રીક્વન્સી 1 હોય તો
        if (frequency[firstCode] > 1) {
          queue.shift();
          continue;
        }

        // જો અહીં પહોંચ્યા તો, આ જ પહેલો યુનિક અક્ષર છે
        if (queue.length == 0) {
          inputS += "-1";
        } else {
          inputS += queue[0];
        }
      }
    }

    // જો કોઈ પણ અક્ષર ન મળે તો -1 પરત કરો
    if (inputS.length == 0) {
      inputS += "-1";
    }

    return inputS;
  );
};
```

મુખ્ય ફેરફાર શું કર્યું?

1. `inputS` અપડેટ કર્યું: `queue.length > 0` ને બદલે `inputS.length == 0` નો ઉપયોગ કરીને ચકાસણી કરી.
   - યુનિકોડ ચારેય કોડની સાથે, કુલ્લ અક્ષરો યુનિકોડ સ્ટ્રિંગમાં નહીં હોવાનું જ્યારે બનશે ત્યારે તેને "-1" `inputS` માં ઉમેરવામાં આવશે.
   - મૂળભૂત ભાષા લોજિક, તેને કોઈ પણ ભાષા માટે (જેમ કે Python, Java, C++) અનુકૂલિત કરવા સરળ.

ઉદાહરણ માટે કામગીરી:

ઇનપુટ: "abcdef" લોજિક

- 'a' ઉમેરાઈ: queue માં 'a'. `inputS` માં 'a'. ફ્રીક્વન્સી 1 છે. `inputS` માં 'a' ઉમેરાઈ.
- 'b' ઉમેરાઈ: queue માં 'a', 'b'. 'a' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'a' દૂર કરો. queue માં 'b'. `inputS` માં 'b' ઉમેરાઈ.
- 'c' ઉમેરાઈ: queue માં 'b', 'c'. 'b' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'b' દૂર કરો. queue માં 'c'. `inputS` માં 'c' ઉમેરાઈ.
- 'd' ઉમેરાઈ: queue માં 'c', 'd'. 'c' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'c' દૂર કરો. queue માં 'd'. `inputS` માં 'd' ઉમેરાઈ.
- 'e' ઉમેરાઈ: queue માં 'd', 'e'. 'd' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'd' દૂર કરો. queue માં 'e'. `inputS` માં 'e' ઉમેરાઈ.
- 'f' ઉમેરાઈ: queue માં 'e', 'f'. 'e' ની ફ્રીક્વન્સી 1 નથી. queue માંથી 'e' દૂર કરો. queue માં 'f'. `inputS` માં 'f' ઉમેરાઈ.

મૂળ ઇનપુટ કામગીરી પૂર્ણ થયા પછી, `inputS` નું અંતિમ મૂલ્ય: "abcdef"

ટીપ્સ અને સુધારાઓ:

બી.ટી.વી. લોજિક:

- 1+2 ઉમેરી Queue માં ઉમેરાઈ છે. `inputS` માં '1' ઉમેરાઈ.
- 1+1 ઉમેરી Queue માં ઉમેરાઈ છે. '1' ની ફ્રીક્વન્સી બદલાઈ. queue માંથી '1' દૂર કરો. `inputS` માં '2' ઉમેરાઈ.
- 1+3 ઉમેરી: '3' ની ફ્રીક્વન્સી 1 થઈ ગઈ. queue ક્યૂ ખાલી. `inputS` માં '3' ઉમેરાઈ. Queue માં ઉમેરાઈ નથી. `inputS` માં '-1' ઉમેરાઈ કારણ `inputS` ખાલી છે.

ક્યૂ માં સુધારો કરવા સંબંધિત આવશ્યકતા અહીં કેવી રીતે કરી શકાય તેનો એક વૈકલ્પિક રીત છે Sliding Window I Queue તે સંબંધિત સ્ટ્રિંગ કામગીરી કરવા તે જે અહીં કરી છે.

You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position.

Return the max sliding window.

Example 1:
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3

Output: [3,3,5,5,6,7]

આ સ્લાઈડિંગ વિંડોના કામગીરી માટેની સંબંધિત રીત છે **Deque (Double-Ended Queue)** નો ઉપયોગ કરવાની છે. આવતી આવી રીત (O(n)) કામગીરી સ્લાઈડિંગ વિંડો સાથે.

અલ્ગોરિધમ ફ્રીક્વન્સી આવતી રીતે Deque કામગીરી (આવી રીત સ્લાઈડિંગ વિંડો સાથે O(n) કામગીરી કરવા માટે)

1. Deque નું કામગીરી કરવા માટે ઉપયોગ કરો. આ Deque માં ઇન્ડેક્સ સ્ટોર કરશે. તે ડિક્રિઝિંગ ઓર્ડરમાં હશે. આ રીતે, આગળનો તત્ત્વ હંમેશા વિંડોમાં મહત્તમ તત્ત્વ હશે.
2. જ્યારે વિંડોમાં કામગીરી થાય, ત્યારે ખાતરી કરો કે Deque માંથી કોઈ તત્ત્વ બાકી નથી જે વિંડોની બહાર છે. આ કરવા માટે, તમારે Deque ના આગળના તત્ત્વ અને વર્તમાન વિંડોની શરૂઆત વચ્ચે તુલના કરવી પડશે.
3. જ્યારે વિંડોમાં નવું તત્ત્વ ઉમેરવામાં આવે, ત્યારે તમારે Deque માંથી તમામ તત્ત્વો દૂર કરવા જોઈએ જે નવા તત્ત્વ કરતાં નાના છે. આ ખાતરી કરશે કે Deque હંમેશા ઘટતા ક્રમમાં રહે.
4. **Result** એરેમાં મહત્તમ તત્ત્વ ઉમેરો: વર્તમાન વિંડોના શરૂઆતી ઇન્ડેક્સ પર પહોંચતા, વર્તમાન વિંડોના મહત્તમ તત્ત્વ (Deque નું આગળનું તત્ત્વ) પરિણામ એરેમાં ઉમેરવામાં આવશે.

Complexity:

- **Time Complexity:** O(n). દરેક તત્ત્વ ફક્ત એક વખત Deque માં ઉમેરવામાં અને બહાર કાઢવામાં આવે છે, જેનો અર્થ છે કે કુલ કામગીરી રેખીય છે.
- **Space Complexity:** O(k). Deque માં મહત્તમ k તત્ત્વો હશે.

શું તમારે આ અલ્ગોરિધમ કોડની સંપૂર્ણ અમલીકરણ અહીં ઇચ્છતા નથી કોઈ ચોક્ક્સ ભાષા (જેમ કે JavaScript અથવા Python) માં?

Generate code snippets, no diagrams, no ASCII, check it

Generate code snippets, no diagrams, no ASCII, check it

Gemini can make mistakes, so double-check it